طريقة تشغيل المشروع ومشاهدة النتيجة

دليل خطوة بخطوة على ويندوز، مع الـ output المتوقع بعد كل أمر

0. قبل أن تبدأ

- جهاز ويندوز 10/11 (أو ماك/لينكس: نفس الأوامر).
- اتصال بالإنترنت **في أول مرة فقط** (لتنزيل المكتبات، حوالي 80 مكتبة).
- مساحة حوالي 500 ميجا (المكتبات + الداتا + الناتج).
- لا حاجة لصلاحيات Administrator.

1. تثبيت `uv` (مرة واحدة)

`uv` هي الأداة التي تنزّل بايثون والمكتبات وتشغّل المشروع. افتح PowerShell والصق السطر التالي كاملاً في سطر واحد:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

ينتهي بعبارة `everything's installed!`. ثم **أغلق الـ Terminal وافتحه من جديد** (أو شغّل في نفس النافذة: `$env:Path = "$env:USERPROFILE\.local\bin;$env:Path"`) وتحقق:

```
uv --version
uv 0.13.4 (...)
```

> الأمر يثبّت أداة من الموقع الرسمي `astral.sh` في مجلد المستخدم فقط (`C:\Users\<اسمك>\.local\bin`) ولا يلمس ملفات النظام. تأكد أن الرابط مكتوب بالضبط كما فوق.

2. فك الضغط والدخول إلى المجلد الصحيح

فك ضغط الـ zip في مكان عادي قابل للكتابة (مثل Downloads أو Documents، وليس `Program Files`). ثم افتح المجلد في VS Code (`File → Open Folder`) وافتح Terminal (`Terminal → New Terminal`).

الأهم: تأكد أنك في المجلد الذي فيه `pyproject.toml`:

```
dir
    dbt    tools    run.py    pyproject.toml    uv.lock    README.md    RUN_ON_WINDOWS.bat ...
```

> إذا ظهرت لك الرسالة `error: No pyproject.toml found in current directory` فأنت في المجلد الأعلى بخطوة واحدة (فك الضغط يعمل أحياناً مجلداً داخل مجلد). اكتب `cd qafila-margin-final` ثم `dir` مرة أخرى.

3. التشغيل السريع بنقرة واحدة

انقر مرتين على `RUN_ON_WINDOWS.bat`. يشغّل الخطوات الخمس التالية بالترتيب، ويتوقف عند أول خطأ ويعرضه، وينتهي برسالة `SUCCESS`. إن فضّلت رؤية كل خطوة بنفسك فاتبع الخطوات اليدوية التالية.

4. التشغيل اليدوي خطوة بخطوة

1 تجهيز البيئة

```
uv sync
```

أول مرة يأخذ دقيقة أو اثنتين، وينتهي بقائمة `Installed 79 packages`. يُنشئ مجلد `.venv` فيه بايثون والمكتبات (duckdb، dbt، pyiceberg...).

2 توليد بيانات التجربة

```
uv run python tools/make_data.py
```

الـ output المتوقع (الأرقام ثابتة لأن الـ seed ثابت):

```
wrote ...\data
  customers 7,442 (snapshot 6,060, 6,346 CDC events)
  products 356 | stores 12 | orders 45,766 | order_items 172,545
  payments 41,785 | delivery events 22,518 | reviews 3,000
```

يُنشئ مجلد `data/` (حوالي 35 ميجا) فيه قاعدة التطبيق وملفات الموردين الأربعة.

3 تشغيل الـ pipeline (هذا هو الأمر الرئيسي)

```
uv run python run.py
```

ينفّذ ثلاث مراحل: يحمّل البيانات إلى DuckDB، ثم يبني موديلات dbt ويشغّل اختباراتها، ثم ينشر الجداول إلى Iceberg. تظهر عشرات الأسطر من dbt (`START` و`OK` و`PASS`) ثم تأكد أن النهاية هي:

```
Done. PASS=34 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=34
  silver.product_cost_history: created, 633 rows
  gold.weekly_margin: created, 165 rows
  quality.price_quarantine: created, 7 rows
done: ...\warehouse
```

> **علامة النجاح:** `ERROR=0` وثلاثة أسطر `created` وسطر `done:` في آخر الشاشة.

> قد يظهر قبلها `WARNING: MissingArgumentsPropertyInGenericTestDeprecation` وبعدها ملخص `DeprecationsSummary`. هذا تحذير من dbt عن طريقة كتابة خاصية في ملفات yml، ولا يؤثر على النتائج ولا

يعني فشلاً.

4 فحص العقد

```
uv run python tools/check.py
```

يجب أن تكون كل الأسطر ✅ وتنتهي بـ:

```
22/22 checks passed
```

5 المقارنة مع الحساب المستقل

```
uv run python tools/reference_check.py
quarantine: expected 7, got 7 -> OK
history: 633 versions for 328 SKUs; SKUs that differ: 0 -> OK
weekly_margin: 165 expected rows, 165 got, rows that differ (>0.02): 0 -> OK
```

هذا السكريبت يحسب النتيجة من الملفات الخام بـ Python عادي (دون SQL) ويقارنها بالجداول المنشورة. إن ظهر `MISMATCH` في أي سطر فهناك مشكلة.

5. أين النتيجة وكيف أراها؟

النتيجة ليست تقريراً مطبوعاً على الشاشة، بل **ثلاثة جداول Iceberg** داخل مجلد `warehouse/`. لعرضها:

```
uv run python tools/show_tables.py
```

يطبع لكل جدول عدد صفوفه وأول صفوفه وقائمة الـ snapshots، وفي النهاية مقارنة «السفر عبر الزمن». مثال مما يعرضه:

```
=== silver.product_cost_history: 633 rows ===
        sku  unit_cost_sar valid_from   valid_to  is_current
QF-BAK-1058           4.16 2026-08-31 2026-09-14       False
QF-BAK-1058           3.87 2026-09-14 2026-09-21       False
...
=== gold.weekly_margin: 165 rows ===
 iso_week   city   category  lines  revenue_sar  costed_revenue_sar  cost_sar  margin_sar
uncosted_lines
       36 Dammam     Bakery    179      1739.23              791.27    544.94      246.33
110
...
=== quality.price_quarantine: 7 rows ===
prices_2026-W39.csv QF-DAI-1006 2026-09-21     -1 invalid_cost
prices_2026-W39.csv QF-NEW-9292 2026-09-21  18.34  unknown_sku
```

الجدول	ماذا يحمل	الأعمدة

`silver.product_cost_history`	نسخ تكلفة كل منتج عبر الزمن (633 صفاً لـ 328 SKU)	`sku, unit_cost_sar, valid_from, valid_to, is_current`
`gold.weekly_margin`	الهامش لكل أسبوع × مدينة × تصنيف (165 صفاً)	`iso_week, city, category, lines, revenue_sar, costed_revenue_sar, cost_sar, margin_sar, uncosted_lines`
`quality.price_quarantine`	صفوف الموردين المرفوضة مع السبب (7 صفوف)	`source_file, sku, valid_from, raw_cost, reason`

مجلدات تظهر بعد التشغيل: `data/` (المدخلات)، `warehouse/` (قاعدة `qafila.duckdb` وفهرس `catalog.db` وملفات Iceberg)، `dbt/target/` (سجلات dbt).

6. إعادة التشغيل: لا يتغير شيء

```
uv run python run.py
  silver.product_cost_history: unchanged (633 rows), no new snapshot
  gold.weekly_margin: unchanged (165 rows), no new snapshot
  quality.price_quarantine: unchanged (7 rows), no new snapshot
```

أي أن التشغيل المتكرر على نفس البيانات آمن ولا يضيف شيئاً ولا يكرّر صفاً.

7. وصول التصحيح المتأخر

```
uv run python tools/make_data.py --add-correction
uv run python run.py
  silver.product_cost_history: updated in place, 637 rows (was 633)
  gold.weekly_margin: updated in place, 165 rows (was 165)
  quality.price_quarantine: unchanged (7 rows), no new snapshot
uv run python tools/reference_check.py
```

الأمر الأول يضيف ملف `prices_2026-W39_fix.csv` (9 صفوف). بعد التشغيل تتحدّث الجدولان **في مكانهما** ويبقى جدول الحجر كما هو (الصفوف السيئة ما زالت في ملف W39 الأصلي). والمقارنة المستقلة تبقى `OK`.

لرؤية «ما قبل وما بعد التصحيح» شغّل مرة أخرى `uv run python tools/show_tables.py` وانظر آخر قسم منه:

```
=== Time travel: gold.weekly_margin totals inside each snapshot ===
  snapshot 6519392441133803612: rows=165  revenue=3,504,002.19  cost=1,654,199.98
margin=801,016.88
  snapshot 7961518883321039657: rows=165  revenue=3,504,002.19  cost=1,651,701.06
margin=803,515.80
```

السطر الأول هو الهامش **قبل** التصحيح (ما زال محفوظاً)، والثاني **بعده**. أرقام الـ snapshots (الأعداد الطويلة) تختلف عندك في كل تشغيل، أما المبالغ فهي نفسها.

8. تتبّع منتج وأوردر بعينك

```
uv run python tools/spot_check.py
```

يختار منتجاً له عدة نسخ تكلفة ويعرض ما قالته ملفات الموردين حرفياً ثم نسخ التاريخ الناتجة، ثم يحسب سطر أوردر واحداً يدوياً (التوقيت، الإيراد، النسخة السارية، التكلفة، الهامش)، ثم يعرض صفوف الحجر. ولمنتج معيّن: `--sku QF-BAK-1058`.

9. تجربة على بيانات أخرى (seed مختلف)

```
uv run python tools/make_data.py --seed 11 --out data_11
uv run python run.py --data-dir data_11 --warehouse wh_11
uv run python tools/check.py --data-dir data_11 --warehouse wh_11
uv run python tools/reference_check.py --data-dir data_11 --warehouse wh_11
```

مجموعة بيانات بأرقام مختلفة، لكن بنفس أنواع المشاكل. تظهر هنا: 632 نسخة تكلفة لـ 314 SKU، و7 صفوف حجر، و165 صفاً في الهامش، وكلها `OK`. وهذا دليل أن الكود لا يعتمد على أرقام أو أسماء ثابتة.

10. المشاكل الشائعة

الرسالة / المشكلة	السبب	الحل
`uv` is not recognized	الـ Terminal لا يعرف مسار `uv` بعد	أغلق الـ Terminal / VS Code وافتحه من جديد، أو شغّل `$env:Path = "$env:USERPROFILE\.local\bin;$env:Path"`
`No pyproject.toml found`	أنت في المجلد الأعلى	`cd` إلى المجلد الذي فيه `pyproject.toml`
`qafila_oltp.sqlite not found`	لم تولّد البيانات	شغّل `uv run python tools/make_data.py` قبل `run.py`
فشل أثناء `uv sync`	الإنترنت أو جدار حماية / proxy	تأكد من الاتصال وأعد `uv sync`
رسالة إصدار بايثون	بايثون خارج 3.10–3.13	`uv` ينزّل الإصدار المناسب تلقائياً؛ وإلا ثبّت بايثون 3.13
خطأ صلاحيات (Permission)	المجلد غير قابل للكتابة أو مزامنة سحابية	انقل المشروع إلى مجلد عادي مثل `Documents`
`dbt build failed`	فشل موديل أو اختبار	اقرأ أسطر `ERROR` / `FAIL` فوقها وأرسلها كاملة لمن يدعمك

خطوط صفراء تحت `import` `duckdb` في VS Code	VS Code لا يستخدم بيئة المشروع	`Ctrl+Shift+P` ثم `Python: Select Interpreter` ثم اختر `.venv`. لا تؤثر على التشغيل

11. البدء من جديد

لإعادة كل شيء من الصفر احذف المجلدات المُولّدة: `data` و`warehouse` و`dbt\target` ثم أعد الخطوات 2 و3.
تنبيه: حذف `warehouse` يمحو تاريخ الـ snapshots (السفر عبر الزمن).

```
Remove-Item -Recurse -Force data, warehouse, dbt\target, dbt\logs
```

12. ملخص الأوامر

الغرض	الأمر
تجهيز البيئة	`uv sync`
توليد البيانات	`uv run python tools/make_data.py`
تشغيل الـ pipeline	`uv run python run.py`
فحص العقد (22/22)	`uv run python tools/check.py`
مقارنة مستقلة	`uv run python tools/reference_check.py`
عرض الجداول والـ snapshots	`uv run python tools/show_tables.py`
تتبّع منتج وأوردر	`uv run python tools/spot_check.py`
إضافة التصحيح المتأخر	`uv run python tools/make_data.py --add-correction`